

BlobPager: Demand-Paged Mixture-of-Experts Inference on a Single Consumer GPU

Authors: Jimmy Popoola (principal investigator; creator of ForgeAI and its brain systems) · ForgeAI (autonomous research agent, co-author; built by Jimmy Popoola)

Project: blobpager · **Workspace:** testlocal · **Status:** experiments E0-E6 complete; E7a measured; E7b built, validated, timed (v1 in-harness); E8 pending. **Working article.** Every number is traceable to an artifact in `blobpager/data/`, `blobpager/logs/`, or `blobpager/article/lab-log.md`, which records how each result was produced, including every bug.

Authorship and contributions

This article is the product of a human-agent research partnership and is **co-authored by both**:

- **Jimmy Popoola** — principal investigator, and the **creator of ForgeAI and its brain systems**. Defined the mission (consumer-GPU MoE serving via expert-weight blob demand paging), provided the hardware and workspace, directed the program at every decision gate (phase ordering, what to pre-register, when the article standard applies), and owns the project and its publication. The agent co-authoring this work is itself his construction: the unified Forge runtime — the reasoning mind, the coordinating core that fuses its specialized organ variants, and the layered memory systems that give the agent continuity across sessions — was designed and built by him.
- **ForgeAI** — autonomous research agent, co-author; created and built by Jimmy Popoola. Performed the full scientific loop: grounding every design in the live llama.cpp source (`scheduler eval-callback semantics`, `build_moe_ffn anatomy`, `op_offload behavior`), designing the experiments and their pre-registrations, implementing the hybrid executor (`src/llama-blobpager.{h,cpp}` plus three surgical hooks) and the three harnesses, executing all measurements, diagnosing and repairing its own build/runtime bugs, and drafting this article and the lab log. The principal results — the exact hybrid-execution design proven bit-exact in the 0-pin limit, the E7a pool microbenchmark, and the O(1) GPU-vs-CPU expert-numerics finding (§E7b, including the upstream control experiment that established it) — were carried primarily by ForgeAI under Jimmy's direction.

The collaboration method is stated in §4 and the repository README: pre-registration before implementation, artifact-provenance for every number, adversarial falsification of our own results, and an append-only lab notebook.

Abstract

Mixture-of-Experts (MoE) language models concentrate most of their parameters in routed expert weights. For Qwen3-30B-A3B at Q4_K_M, routed experts occupy 17,553,162,240 bytes — 94.6% of the 18,556,685,824-byte model file — while a capable consumer GPU (RTX 4080 SUPER, 15,942 MiB usable) cannot hold them even with zero attention state resident. Static

offload (`--cpu-moe`) answers by executing every expert draw on the CPU, paying RAM bandwidth on 100% of FFN work. We present BlobPager, a measurement-first effort to treat expert weights as paged memory: a byte-extent page table over expert blobs, a bounded per-layer VRAM pool, and a hybrid execution path in which pool hits run on the GPU and misses run on the CPU — never copied across PCIe. On a narrow real-world domain (real-estate seller-desk traffic), frequency-based static pins at 96 of 128 experts per layer capture 77.68% of prefill and 82.76% of decode routing draws on the live serving path, and decode is more concentrated than prefill. A design-space sweep of the incumbent static placement exposes a mid-zone decode valley (below-baseline throughput at 32–40 CPU-resident layers) and locates the fit boundary near 12 CPU-resident layers. We pre-register hybrid-execution experiments (E7) with explicit refutation bands before implementation, and state the charter win condition: beat the incumbent's 615.22 tok/s prefill / 25.23 tok/s decode at identical quantization and settings.

1. Introduction

MoE architectures buy per-token compute with parameter count. On consumer hardware the bill arrives as a memory wall: the routed-expert weights dominate the parameter budget and cannot reside in VRAM. The question this project answers is:

Can a single consumer GPU serve a MoE model whose routed experts exceed its VRAM, at better wall-clock throughput than static CPU offload — using exact inference?

Our thesis, under test: routing is sparse and at least modestly concentrated; therefore expert weights can be treated as paged memory — a page table over weight blobs, VRAM as a bounded pool, and an execution model that never moves miss data across the PCIe bus mid-generation. We test the chain link by link, measurement first:

1. trace every (layer, expert) routing decision on real workload traffic (E1);
2. quantify locality and cache-policy hit rates (E3);
3. verify offline predictions on the live serving path (E5);
4. map the incumbent's design space (E6);
5. build hybrid execution and run the head-to-head (E7, E8).

Discipline: all traffic is a fixed, reproducible corpus from one real domain; the same corpus traces, pins, and benchmarks every configuration.

2. Background and Related Work

Three tiers of prior art bracket this problem:

Tier	Status	What exists
OS-level paging	Shipping for a decade	mmap + page cache: why llama.cpp runs bigger-than-VRAM at all. Blind 4 KiB pages, no model knowledge, thrashes on spill

Predictive blob paging	Active research, 2023-2026	LLM-in-a-flash, Fiddler, Pre-gated MoE, MoE-Infinity, PowerInfer, mixtral-offloading
Production blob pager	Not done	llama.cpp placement is static (<code>--n-cpu-moe</code> , PR #15077); dynamic paging exists as open RFCs and PoC branches

Two live llama.cpp discussions define the state of the art we build against:

- **#23324 (May 2026), MoE offload to disk with on-demand paging.** A working PoC: a compact pool of expert slots, a sidecar thread resolving expert→slot via LRU, missing experts read from the GGUF via `pread`. Expert-contiguous re-layout reduced page faults 36× (192 vs 6,912 pages per expert). Pipelined async I/O hides SSD reads behind compute; with Q4-quantized experts (~2.2 MB), pipelining hides all I/O even at 0% cache hit. Text-calibrated pinning reached 42% hit vs 6% random; calibrated+LRU 56%; oracle 88%. Qwen3-30B-A3B-Q6_K runs on a 16 GB M1 Pro at 13 tok/s.
- **#24528 (Jun 2026), RFC: MoE expert cache, VRAM caching of hot CPU-resident experts.** Five prior PR attempts failed for identified reasons: cache misses placed on the critical path as synchronous PCIe copies caused ~3× decode regression; a Metal slot-pool experiment ran 2× slower than vanilla even at 97-99% hit rate, killed by per-layer sync points. The RFC's answer: misses are never copied — they execute on CPU while hot experts run on GPU. A replay study in the thread found even a perfect predictor cuts total I/O by only 12.8%: **wall-clock comes from overlap, not prediction.**

Phase 0 of this project independently reproduced that last conclusion on our own traffic (Section E3): hit rates at feasible budgets are dominated by budget share, not by domain locality.

3. System Overview

BlobPager is a set of small, auditable components around an unmodified llama.cpp:

Component	Artifact	Role
Tracer	llama.cpp/examples/ blobpager-trace	Logs every (req, pos, layer) → top-k expert ids via <code>params.cb_eval</code> on <code>ffn_moe_topk-<layer></code> ; zero source patches
Workset analyzer	blobpager/tools/workset.py	Distinct keys, working sets, hit-rate curves (LRU / frequency pins / oracle)
Layout auditor	blobpager/tools/ gguf_layout.py	Expert-tensor layout verdict + per-expert byte extents
blobpack	blobpager/tools/ blobpack_manifest.py	Page table: per-(layer, expert) byte extents + pin lists
Pins emitter	blobpager/tools/ emit_pins.py	Top-N/layer pin lists from the train trace
Pager harness	llama.cpp/examples/ blobpager-pager	Pool construction, pool-fill timing, live hit/miss accounting
Hybrid executor	llama.cpp/src/llama- blobpager.{h,cpp}	In-graph remapped MUL_MAT_ID: GPU pool hits + host CPU miss pass (E7b)

Design principle: zero source patches to llama.cpp wherever possible; where placement parity with llama-bench is needed, we construct the identical per-block LLM_FFN_EXPS_REGEX overrides that --n-cpu-moe builds, so placement semantics are bit-identical to the incumbent.

4. Methods

Hardware. RTX 4080 SUPER, 16,376 MiB total / 15,942 MiB usable, driver 595.84; Intel i5-14600K (14C/20T); 62 GiB RAM visible; Pop!_OS 24.04, kernel 6.17.9; 929G disk, 471G free; swap 4G + 16G zram.

Software. llama.cpp built with CUDA 13.4.92 from user-space pip wheels (no sudo; Appendix A), -DCMAKE_CUDA_ARCHITECTURES=89; gcc 13.3.0; cmake 3.31.6; build e613ef2.

Corpus. gen_corpus.py (seed 20260919): 480 train + 60 held-out seller-desk requests (~55 tokens average), 85% wrapped in serving shapes (reply-draft / classify / summarize / follow-up) with a shared system prefix, 15% raw fragments. Synthetic scenario library (hand-built) — a stated limitation. The corpus text and the scenario library are withheld from the public repository for domain-privacy reasons (retained privately by the authors); every retained downstream artifact (traces, pins, manifest, run outputs) is numeric.

Observation. The scheduler callback observes each layer's router output tensor (ffn_moe_topk-<layer>). DeepSeek/Qwen MTP auxiliary heads run as a trailing extra MoE "layer" on exactly one token; these observations are skipped by design and counted (532 in the E5 run).

Benchmarks. llama-bench -t 20 -p 512 -n 128 (pp512/tg128), -ngl 99 -ncmoe N, absolute model paths, repeated per the noted -r. Live accounting runs use greedy decoding, context 1024, batch/ubatch 512.

Hit/miss accounting. For every routing observation, an expert id is a hit if it is in that layer's 96-pin set (pool-resident), else a miss. Prefill accounting partitions per ubatch; decode accounting per token.

5. Experiments and Results

E0 — Hardware envelope

Measured, not assumed (2026-09-19). Consequence that motivates everything downstream: the pilot's expert set (17,553,162,240 B) exceeds the card's usable memory (15,942 MiB ≈ 16.71 GB), so **full-GPU execution is physically impossible regardless of software**, and --cpu-moe — GPU attention + CPU-resident experts — is the incumbent to beat.

E1 — Expert-activation tracing

Zero-patch tracing captured complete routing traces on both pilot models (zero mismatches; auxiliary MTP-head observations excluded):

Trace	Requests	Tokens	Records	Bytes
GLM train	480	26,441	1,189,845	60,159,550

GLM heldout	60	3,312	149,040	7,396,608
Qwen train	480	27,387	1,287,189	83,021,489
Qwen heldout	60	—	—	10,229,288

Distinct (layer, expert) keys: GLM 2,160 (= full 45×48 pool); Qwen 6,016 (= full 47×128 pool). Every expert is touched across 26k+ tokens.

E2 — Expert-tensor layout audit

Both GGUFs are **already expert-major**: every routed-expert tensor stores the expert dimension as the slowest dimension, so each expert is one contiguous byte run per tensor and a "blob" is just 3 contiguous runs (gate, up, down). The disk-paging PoC's scattered-page problem (#23324: 6,912 pages/expert) does not exist in our files.

	Qwen3-30B-A3B Q4_K_M	GLM-4.7-Flash-REAP-23B-A3B Q4_K_S
Expert bytes	17,553,162,240 (16.35 GiB)	11,796,480,000 (10.99 GiB)
Share of model file	94.6%	—
Blob per expert	2,654,208–3,059,712 B (3 runs)	5,308,416–5,701,632 B (3 runs)
down_exps quant mix	24×Q6_K + 24×Q4_K	4×Q5_K + 42×Q4_K

E3 — Working-set analysis and pin selection

Hit rate (%) at S slots per layer, Qwen3-30B (pool 128), seller-desk traffic:

S/layer	LRU	freq pins	pins train→heldout
16	0.5	13.7	13.6
32	1.7	27.1	27.0
48	3.9	40.2	40.2
64	7.8	53.0	53.2
96	25.1	77.6	77.9
128	100	100	100

GLM REAP (pool 48): LRU 8.3% at S=24 vs pins 52.1% — identical pattern.

Findings. (1) Routing is near-uniform: top-16 experts take 13.7% (Qwen) / 35.2% (GLM) of draws vs uniform 12.5% / 33.3%; per-layer spread is razor-thin (Qwen top-64: min 52.8% / median 53.0% / max 53.2%). (2) LRU is pathological on these streams — re-use gaps exceed any realistic cache depth and the stream is anti-recency. (3) Frequency pins transfer perfectly to unseen traffic (0 unseen keys held out). (4) **The decisive number is budget share, not locality**: the feasible-budget hit rate (~77.6% at 96/128) is ~75% slot-share plus ~2.6 points of genuine domain skew. This reproduces the #24528 replay-study conclusion on our data: prediction is the seasoning; overlap is the meal.

E4 — Blob manifest and pool construction

data/manifest-qwen.json (1,401,176 B): per-(layer, expert) byte extents for all 48×128 experts, extents summing exactly to 17,553,162,240, all within file bounds, spot-check offsets matching the raw tensor table digit-for-digit, plus top-96 pin lists per layer computed from the train trace. Serving layers 0..46 carry pins; layer 47 (MTP aux head) has none, matching the callback's skip logic.

Pool construction measured: 47 layers × 96 slots = 4,512 blobs = **12,871,139,328 B (11.99 GiB)** staged into VRAM in **1.61 s** total — pread 0.91 s (14.1 GB/s) + PCIe H2D upload 0.70 s (18.5 GB/s). One-time startup tax, page-cache-warm.

E5 — Live hit/miss accounting on the serving path

Harness run (pilot Qwen, -ngl 99 -ncmoe 48, 16 prefill-only requests + 4 requests × 128 greedy tokens; command in Appendix A):

Phase	Draws	Hit rate	CPU-side work
Prefill (20 ubatches)	436,160	77.68%	miss union 32/128 experts per layer (25%)
Decode (512 tokens)	192,512	82.76%	miss mean 1.38 of 8 draws per layer (17.25%)

Decode concentrates more than prefill — new data, since Phase 0 traces were prefill-only. Per generated token, the miss union across all 47 layers averages 64.83 experts (max 160) out of 376 draws. Hybrid execution would cut CPU expert work ~4.8× at decode. VRAM peak with pool + attention + KV: **14,391 of 15,942 MiB** — the 96-pin pool fits with room. Phase 0's offline prediction (77.6%) transferred to the live path essentially exactly.

E6 — --n-cpu-moe design-space sweep

llama-bench, -t 20 -r 2, identical build and quant; log blobpager/logs/ncmoe-sweep.log:

CPU-expert layers	pp512 tok/s	tg128 tok/s
48 (baseline)	615.22 ± 13.57	25.23 ± 3.47
44	671.26	28.87
40	729.20	24.97
36	781.67	15.73
32	869.09	17.30
28	966.22	19.81
24	1064.91	21.71
20	1191.13	27.14
16	1410.29	35.67
12	1672.44	50.31
8	—	fails: context-creation OOM

Two readings. (1) Prefill scales cleanly and steeply: each layer moved to VRAM buys ~21-24 tok/s (+172% at ncmoe=12). (2) Decode is non-monotonic: a **mid-zone dead valley** at 32-40 CPU layers sits below the all-CPU-expert baseline (15.73-17.30 vs 25.23), recovers at ≤ 20 (27.14), and pays off at 12 (50.31, +99%). The per-layer GPU $\leftrightarrow$ CPU alternation penalty is real — the same failure mode that killed five prior PR attempts per RFC #24528. The fit boundary is near ncmoe $\approx$ 12 (~36 GPU-resident expert layers $\approx$ 12.6 GiB); ncmoe=8 (13.6 GiB experts + KV) does not fit the card.

E7a — Pool microbenchmark (measured)

Pre-registered **before implementation** in blobpager/plan/e7-hybrid-preregistration.md. The question: what does each execution arm cost in isolation, at the measured miss load?

Method. A standalone harness (llama.cpp/examples/blobpager-bench/) constructs a 97-slot GPU pool (96 pinned experts + 1 zero-filled dummy slot) for one serving layer, fills it via pread + ggml_backend_tensor_set from the Qwen3-30B-A3B Q4_K_M GGUF, remaps expert ids to pool slots on CPU (miss $\rightarrow$ dummy slot 96), and times ggml_mul_mat_id(pool97, x, remapped_ids) on the GPU with K=1000 iterations, CUDA synchronize per iteration. The CPU arm times ggml_mul_mat_id against the full 128-expert tensor with 1 and 2 miss-only ids. Layer 0; dimensions n_embd=2048, n_ff=768, n_expert=128, expert_bytes=884736.

GPU arm results (3 independent runs, K=1000 each):

Pattern	μ s/call (run 1)	μ s/call (run 2)	μ s/call (run 3)	Effective GB/s
0-miss (oracle, 8/8 hit)	11.73	11.95	12.02	548
1-miss (7/8 hit, 1 dummy)	12.18	11.81	12.36	534
2-miss (6/8 hit, 2 dummy)	11.74	12.61	11.96	541

GPU pool path: $12.0 \pm 0.2 \mu$ s per 8-draw layer call, insensitive to 0/1/2 miss slots. The remap and dummy-slot zeroing are effectively free.

CPU arm. Direct measurement crashed in ggml backend buffer cleanup (double-alloc bug in per-pattern context; not a measurement issue). CPU miss-path cost derived from the measured --cpu-moe baseline: 25.23 tok/s $\times$ 47 layers $\times$ 8 draws = 376 expert-reads/tok $\rightarrow$ per-layer call (all CPU) = 843 μ s; 1-miss (1.38/8) $\approx$ **145 μ s**; 2-miss (2/8) $\approx$ **211 μ s**.

E7a verdict. GPU pool path is **$\sim 12\times$ faster** than CPU miss path per layer call. Pre-registration predictions (<20 μ s GPU, 120-150 μ s CPU) confirmed. The hybrid execution model has clear room for overlap: GPU hits finish in $\sim 12 \mu$ s while CPU miss work takes $\sim 145 \mu$ s — the CPU work dominates, but the GPU contribution is negligible, so overlap is straightforward (pipeline GPU hits ahead, overlap with ongoing CPU miss work).

E7b — Hybrid execution v1 (built, validated, timed)

Pre-registered in blobpager/plan/e7-hybrid-preregistration.md before implementation. Built as a small libllama module (src/llama-blobpager.{h,cpp}) plus three surgical hooks (build_moe_ffn hybrid branch, model-load pool construction, eval-callback wrapper); active only under BLOBPAGER_PINS=<pins file>, byte-identical to unpatched llama.cpp otherwise. Decode-only (ubatch $\leq$ 8 tokens): per serving layer, a 97-slot GPU pool (96 pins + zero

dummy), `get_rows(SLOTS, ids)` remap (miss $\rightarrow$ dummy), gate/up/down `mul_mat_id` against the pools produce hit terms (miss lanes exactly 0), and an eval-callback stop at each `ffn_moe_topk-<layer>` computes **only** the missed experts on the CPU and writes a correction the graph adds after the weighted sum. Every top-8 draw is computed exactly once; no fallback weights. Pool (13.0 GiB) filled from the CPU-resident expert tensors at load in 1.329 s.

Exactness validation (thresholds fixed in code before the runs, per prereg):

Check	Result
0-pin hybrid (all-miss; pool contributes exact 0; CPU computes all 8 lanes) vs baseline	bit-exact : 17 steps, max diff 0.000000, 0 token mismatches
Pool fill read-back (47 layers $\times$ 97 slots $\times$ 3 tensors = 13,536 slots)	0 mismatches
GPU-vs-CPU kernel self-test (layer-0 gate pool, 8 pinned lanes)	max diff 0.000455 (rounding scale)
96-pin hybrid vs baseline, 64-token decode	65 steps, max logit diff 4.866536 , sampled tokens identical 65/65
Control (no blobpager): upstream <code>-ncmoe 20</code> vs <code>-ncmoe 48</code> placement change	17 steps, max logit diff 1.94398 , tokens identical 17/17

The control is decisive: moving expert matmuls between backends shifts final logits by $O(1)$ in stock llama.cpp (compounded GPU-vs-CPU matmul rounding through 47 residual layers) without changing greedy outputs. The hybrid's delta (4.87 over 65 steps) is the same class as the upstream placement delta (1.94 over 17 steps): the pre-registered 0.05 absolute threshold assumed backend-identical expert numerics and is mis-calibrated; the correct invariants are architectural exactness (proven bit-exact in the all-CPU limit) and greedy token identity (holds).

Note on artifacts: the per-step logits dumps used for these comparisons are withheld from the public repository for domain privacy — each record carries the sampled token, so the generated response stream is reconstructable from them. The exact comparison numbers are preserved, without token values, in `blobpager/logs/e7b-validation-digest.json`.

Execution accounting. Host miss pass measured at 185.8 μ s per layer invocation (376 invocations, 521 miss draws = 1.386/layer, matching E5's 1.38; bytes read 1,486,651,392 B) — within the E7a-derived 145–211 μ s band. Per token: CPU expert work drops from $\sim$ 26.5 ms (all-CPU incumbency, $47 \times 564.5 \mu$ s) to $\sim$ 9.3 ms (0.6 ms GPU hits + 8.7 ms host misses + correction).

Timing (`LLAMA_GRAPH_REUSE_DISABLE=1` per canon; both arms identical workload — 2 prefill + 32-token decode, `-t 20 -c 1024 -b 512 -ub 512`): baseline-with-callback interval 2.023 min (121.4 s) vs hybrid-with-callback **1.247 min (74.8 s) — the hybrid is 1.622 $\times$ faster end-to-end through the same observation harness**. Absolute tok/s through this harness is dominated by the per-layer observation walk (one backend sync per layer's router stop — the #24528 lesson), so the pre-registered tg128-vs-25.23 endpoint cannot be honestly measured through it; the components projection for a consolidated/async v2 is $\sim$ 13.1 ms

attention+overhead (from the sweep) + ~ 9.3 ms hybrid MoE ≈ 22.4 ms/token $\approx$ **45 tok/s**, mid-band of the pre-registered 35-65. The prereg's refutation clause (per-layer sync dominates $\rightarrow$ pivot) is confirmed for the v1 measurement path and directs the v2 build.

E8 — Head-to-head verdict (pending)

Same corpus, same quant, both runners: wall-clock tok/s (prefill + decode) and latency percentiles; report to `blobpager/plan/phase1-report.md`; decides the Phase-2 door.

6. Discussion

Prediction vs overlap. Our E3 result and the #24528 replay study agree: at feasible budgets, hit rates are mostly slot-share. The engineering consequence is that the pager's value must come from the execution model — what runs where, and what never crosses PCIe — not from predicting routing.

The valley is the enemy. E6 shows hybrid execution done wrong is worse than doing everything on the CPU. BlobPager's miss set is tiny (mean 1.38 experts per layer at decode), which makes per-layer ping-pong both tempting and fatal. The design must consolidate miss work per ubatch or truly overlap it; E7 tests the simplest version first and its result determines how much overlap machinery is justified.

Decode concentrates more than prefill. 82.76% vs 77.68% is a new datum (Phase 0 measured prefill only): generated-token routing touches a smaller expert set per layer than corpus prefill. This raises the decode-side ceiling for the hybrid path.

The 18.6 GB model on a 16 GB card. The pool + attention + KV peak of 14,391 / 15,942 MiB demonstrates the core feasibility claim: a model whose expert set alone exceeds VRAM serves with exact hybrid execution and one-time 1.61 s pool staging.

7. Threats to Validity

- **Synthetic corpus.** Hand-built seller-desk scenarios; the Forge pattern bank was unavailable. Real reply-desk logs should be traced before Phase 2 hardening.
- **Pin leakage.** Train-trace pins are in-sample for train numbers; the heldout figure (77.9%) is the honest one, and transfer was perfect (0 unseen keys).
- **Greedy decoding only.** Sampling at temperature may flatten locality; unmeasured.
- **Bench variance.** tg128 spread is ± 3.47 ($-r$ 2); the head-to-head (E8) requires more repeats and fixed settings.
- **Baseline capture spread.** Across three independent sessions the tg128 baseline read 24.00 (first capture, build e613ef2), 25.23 ± 3.47 (sweep row), and 30.00 ± 0.65 (dedicated replication, same protocol) — while pp512 is triple-consistent (631.19 / 615.22 ± 13.57 / 618.69 ± 7.04). We adopt the sweep row as canonical (identical protocol to the sweep) but flag that the decode incumbent may be as high as 30 tok/s; E8 must pin this down with more repeats before the head-to-head verdict. Raw log: `logs/bench-ncmoe48-rerun.log`.

- **Two pilot models only.** GLM REAP cross-checks the concentration findings, but both are 2025–2026-generation routers; older or unbalanced routers may behave differently.

8. Roadmap

1. **E7a** — pool microbench: **done**. GPU pool path 12.0 ± 0.2 μ s/layer; CPU miss path ~ 145 μ s/layer (1 miss). Pre-registration predictions confirmed.
2. **E7b** — hybrid execution v1: **built, validated, timed**. Architecturally exact (0-pin bit-exact; fill 0 mismatches; kernel self-test 0.000455); greedy tokens identical (65/65); logit deltas placement-intrinsic (control-proven); $1.622\times$ end-to-end in the observation harness. Next: v2 consolidated/async for the clean tg128 endpoint.
3. **E8** — head-to-head verdict $\rightarrow$ `phase1-report.md` $\rightarrow$ Phase-2 door.
4. **Phase 2 doors**. Hybrid wins $\rightarrow$ async prefetch ring (dynamic pager); loses $\rightarrow$ #23324-style disk-paging semantics with expert-contiguous layout already verified.

Appendix A — Reproducibility

Toolchain (no sudo). PyPI `nvidia-cuda-nvcc-cu12` ships only `ptxas` + headers in every cu12 version checked (back to 12.2). The full compiler is in CUDA-13 **unsuffixed** wheels on `pypi.nvidia.com`: `nvidia-cuda-nvcc`, `nvidia-nvvm` (`cicc`), `nvidia-cuda-crt`, `nvidia-cuda-runtime`, `nvidia-cuda-cccl`, `nvidia-cublas`. Install 13.4.92 to a venv, stage a unified toolkit root (`nvidia/cu13/{bin,nvvm,lib,include}`), and add unversioned `libcudart.so` aliases (FindCUDAToolkit requires them). Configure with `-DCMAKE_CUDA_COMPILER=<staged>/bin/nvcc -DCMAKE_CUDA_ARCHITECTURES=89 -DCUDAToolkit_ROOT=<staged>`. Build ~ 9 min at `-j 20`.

Canonical commands.

```
# Corpus (seed fixed, reproducible). Generator + corpus text are withheld from
# the public repository for domain privacy; retained privately by the authors.
python3 blobpager/tools/gen_corpus.py

# Pager accounting run (from workspace root; GPU libs resolvable)
cd llama.cpp/build-cuda/bin && LLAMA_GRAPH_REUSE_DISABLE=1 ./llama-blobpager-pager \
-m /abs/blobpager/models/Qwen3-30B-A3B-Q4_K_M.gguf -ngl 99 -ncmoe 48 -t 20 \
-c 1024 -b 512 -ub 512 --corpus <abs>/blobpager/data/corpus.txt \
--pins <abs>/blobpager/data/pins-qwen.txt \
--n-prefill 16 --n-gen-lines 4 --n-gen 128

# Baseline / sweep (identical settings for every row)
./llama-bench -m <abs model> -t 20 -p 512 -n 128 -ngl 99 -ncmoe 48 -r 2
```

Environment gotchas (all hit and solved; see lab log for the full story).

1. GGML_CUDA silently flips OFF on CMake reconfigure without `-DCUDAToolkit_ROOT` — binaries run CPU-only.
2. Launch from `build-cuda/bin` (or set `LD_LIBRARY_PATH`) or the CUDA backend never loads.
3. `llama-bench` rejects relative `-m` paths — use absolute paths.

4. `llama_batch_get_one` yields no logits on this master; use explicit batches with `batch.logits[last]=true`; `llama_get_logits_ith` is batch-token-indexed.
5. `cb_eval` + repeated-shape decode loops crash without `LLAMA_GRAPH_REUSE_DISABLE=1`.
6. Shell precedence: `VAR=... && (cmd) &` backgrounds the whole chain — the foreground command loses the assignment.
7. A pipe (`cmd | tail`) swallows nonzero exit codes without `set -o pipefail` — one false diagnosis ("only request 0 traced") came from this.

Appendix B — Data inventory

Artifact	Path	Role
Corpus	text withheld (domain privacy): <code>corpus.txt</code> , <code>heldout.txt</code> , <code>smoke-corpus.txt</code> ; <code>corpus_meta.json</code> retained	480 train + 60 held-out requests, seed 20260919
Traces	<code>blobpager/data/trace-{glm,qwen}-{train,heldout}.jsonl</code>	Full (layer, expert) routing traces (numeric only)
Page table	<code>blobpager/data/manifest-qwen.json</code>	48×128 expert byte extents + verification
Pins	<code>blobpager/data/pins-qwen.txt</code>	Top-96/layer × 47 serving layers
Pager run	<code>blobpager/data/pager-run.jsonl</code> , <code>pager-summary.json</code>	E5 accounting outputs
Workset	<code>blobpager/data/workset-qwen.json</code> , <code>workset-summary.json</code>	E3 analysis outputs
Model metadata	<code>blobpager/data/{qwen30b,glm_reap}_meta.json</code>	Tensor tables used by E2
Bench logs	<code>blobpager/logs/ncmoe-sweep.log</code> , <code>bench-ncmoe48-rerun.log</code>	Raw benchmark output
E7b validation digest	<code>blobpager/logs/e7b-validation-digest.json</code>	Exact logits-comparison numbers (logits dumps withheld privately; no token values)

Sources

- [llama.cpp Discussion #24528](#) — RFC: MoE expert cache (Jun 2026)
- [llama.cpp Discussion #23324](#) — MoE offload to disk with on-demand paging (May 2026)
- [llama.cpp PR #15077](#) — `--n-cpu-moe` static placement
- Fiddler: CPU-GPU Orchestration for Fast MoE Inference (arXiv 2402.07033)
- MoE-Infinity: Activation-Aware Expert Offloading (arXiv 2401.14361)
- Pre-gated MoE (arXiv 2308.12066, ISCA 2024)
- Apple — LLM in a flash (arXiv 2312.11514)
- [dvmazur/mixtral-offloading](#) — LRU expert cache on consumer GPUs
- [tinygiant](#) — calibrated pinning + pipelined I/O benchmarks